Date 6 March 2026

Project Name: UCab - Modern Ride-Sharing Platform

Document: Final Project Report

1. INTRODUCTION

1.1 Project Overview

UCab is a comprehensive, full-stack ride-sharing web application built using the MERN stack (MongoDB, Express.js, React.js, Node.js). It serves as a bridge between passengers looking for reliable transport and drivers seeking fair earnings. The platform features secure role-based authentication (JWT), real-time ride tracking infrastructure (Socket.io), cost-effective open-source mapping (Leaflet.js), and a direct Peer-to-Peer (P2P) UPI QR-code payment interface.

1.2 Purpose

The primary purpose of UCab is to demonstrate a scalable Minimum Viable Product (MVP) for the ride-hailing industry that eliminates the need for expensive proprietary APIs and high-commission payment gateways. By utilizing open-source tools and a zero-commission business model approach, UCab aims to provide cheaper rides for customers and higher take-home pay for drivers.

2. IDEATION PHASE

2.1 Problem Statement

- **PS-1 (Passengers): Commuters face long waiting times, frequent ride cancellations, and unpredictable surge pricing due to poor driver-matching algorithms in existing apps.**
- **PS-2 (Drivers): Cab drivers lose a massive chunk of their earnings (25-30%) to platform commissions and waste fuel on empty "dry runs" due to inefficient ride allocations.**
- **PS-3 (Safety): Late-night travelers feel unsafe due to laggy GPS tracking and lack of transparent, real-time location sharing.**

2.2 Empathy Map Canvas

(Textual representation for a 4-quadrant box)

- **Says: "I just want a cab that doesn't cancel on me." / "The platform takes too much commission from my daily earnings."**
- **Thinks: "Is this ride safe?" / "Will I reach the office on time?" / "Am I getting paid fairly for this long trip?"**

- **Does: Constantly switches between multiple cab apps to compare prices. / Drivers often call passengers to ask for the drop location before deciding to go.**
- **Feels: Frustrated by high prices, anxious about safety, and exploited by high platform fees.**

2.3 Brainstorming

During the brainstorming phase, the team evaluated various technologies. To solve the issue of high operational costs, we decided to replace Google Maps API with Leaflet.js (an open-source map library). To solve the issue of payment gateway commissions, we brainstormed a Static UPI QR Code approach where passengers pay drivers directly via UPI apps, bypassing the platform's financial holding.

3. REQUIREMENT ANALYSIS

3.1 Customer Journey Map

1. **Awareness/Onboarding: User registers as a Rider or Driver via the custom form.**
2. **Booking: Rider enters Pickup and Drop-off locations and selects a cab type (Mini, Sedan, SUV).**
3. **Payment: Rider scans the static QR code (qrcode.png) via a UPI app and clicks "Payment Completed".**
4. **Tracking: Rider views the assigned driver's details and tracks the ride status.**
5. **Post-Ride: Rider views the completed trip in the Ride History dashboard.**

3.2 Solution Requirement

- **Functional Requirements: User Authentication (JWT), Ride Booking System (Location input, Cab selection), Payment Interface (Static QR), Real-Time Tracking Infrastructure (Socket.io), Dashboard & History.**
- **Non-Functional Requirements: Usability (Bootstrap UI), Security (Bcrypt.js password hashing), Performance (Vite for fast rendering), Scalability (Stateless JWT architecture).**

3.3 Data Flow Diagram

(Textual description to aid in drawing the DFD)

- **Level 0 (Context Diagram): The User (Rider/Driver) interacts with the "UCab System". The system outputs Ride Details, Fare, and Tracking Info back to the user.**
- **Level 1 DFD:**
    - **Process 1: Authentication (Validates credentials with MongoDB).**

- Process 2: Ride Booking (Calculates fare based on cab type).
- Process 3: Payment (Displays QR code, updates status to 'Paid').
- Process 4: Live Tracking (Socket.io broadcasts location updates between Driver and Rider).

3.4 Technology Stack

- Frontend: React.js, Vite, Bootstrap, CSS, Leaflet.js (Maps).
- Backend: Node.js, Express.js, Socket.io (WebSockets).
- Database: MongoDB (Mongoose ODM).
- Security: JSON Web Tokens (JWT), Bcrypt.js.

4. PROJECT DESIGN

4.1 Problem Solution Fit

UCab perfectly fits the identified problems by providing an open-source alternative to expensive ride-hailing architectures. By using Leaflet.js and static QR payments, the operational cost drops significantly, allowing the platform to theoretically operate on a zero-commission model, which directly solves the driver's earning problem and the rider's high-fare problem.

4.2 Proposed Solution

UCab is a full-stack web application featuring secure JWT authentication, role-based dashboards, and real-time communication. It integrates Leaflet.js for cost-effective map rendering and utilizes a P2P UPI QR Code system for payments, drastically reducing platform overhead costs.

4.3 Solution Architecture

The architecture follows a standard 3-tier Client-Server model:

- Presentation Layer (Client): React.js SPA handling UI, Leaflet maps, and Socket.io-client.
- Application Layer (Server): Express.js REST API handling business logic (Auth, Booking) and a Socket.io server handling real-time WebSocket connections (join_ride, update_location).
- Data Layer (Database): MongoDB storing Users, Drivers, and Ride History documents.

5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning

- **Sprint 1 (Setup & Auth): UI setup with Vite/Bootstrap, User Registration & Login APIs, JWT implementation.**
- **Sprint 2 (Booking & Maps): Ride booking UI, Cab selection logic, Leaflet.js map integration.**
- **Sprint 3 (Payment & Tracking): Static QR code payment interface, Socket.io backend infrastructure setup.**
- **Sprint 4 (Dashboard & Testing): Ride history integration, UAT testing, bug fixing, and final deployment.**

(Average Team Velocity: 1.14 Story Points per day).

6. FUNCTIONAL AND PERFORMANCE TESTING

6.1 Performance Testing

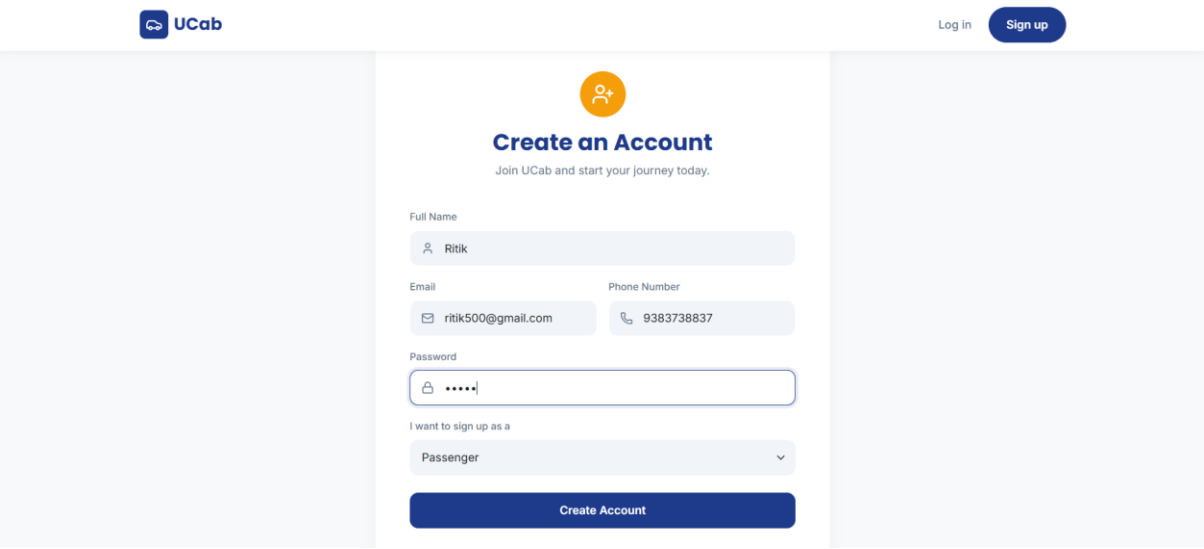
- **UAT Execution: A total of 51 test cases were executed across modules.**
- **Defect Analysis: 30 bugs were fixed (UI glitches, routing errors). 3 bugs were marked as "Won't Fix" (related to actual payment gateway APIs, as the project uses a static QR approach for the MVP).**
- **Performance: The Vite-powered frontend achieved a load time of under 1.5 seconds. The Socket.io WebSocket connection maintained stable, low-latency communication during simulated ride tracking.**

7. RESULTS

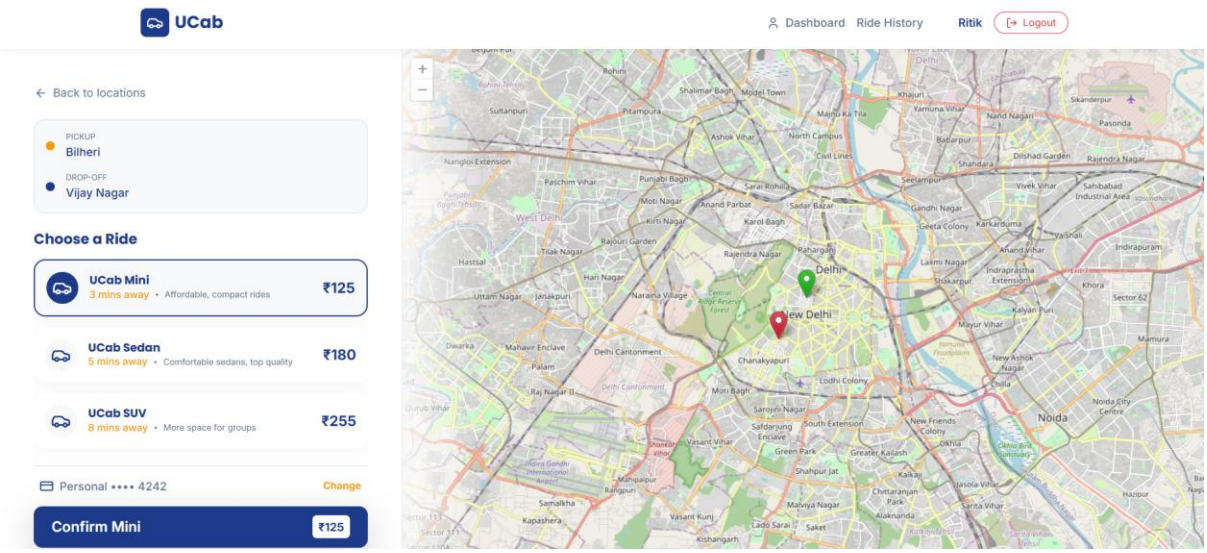
7.1 Output Screenshots

(Note: Insert your application screenshots below)

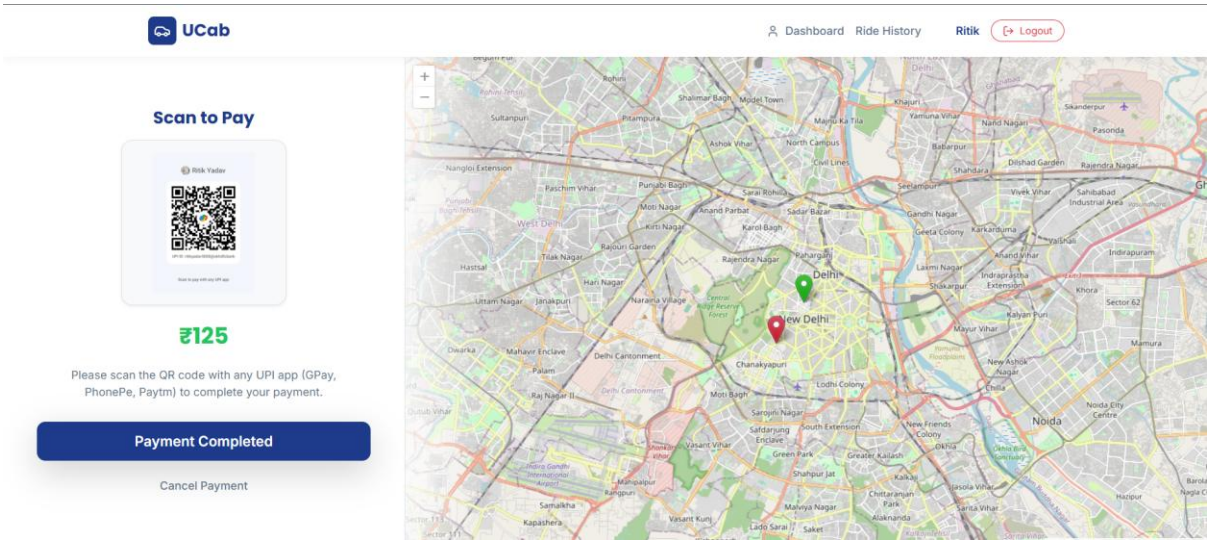
[Login / Registration Page]



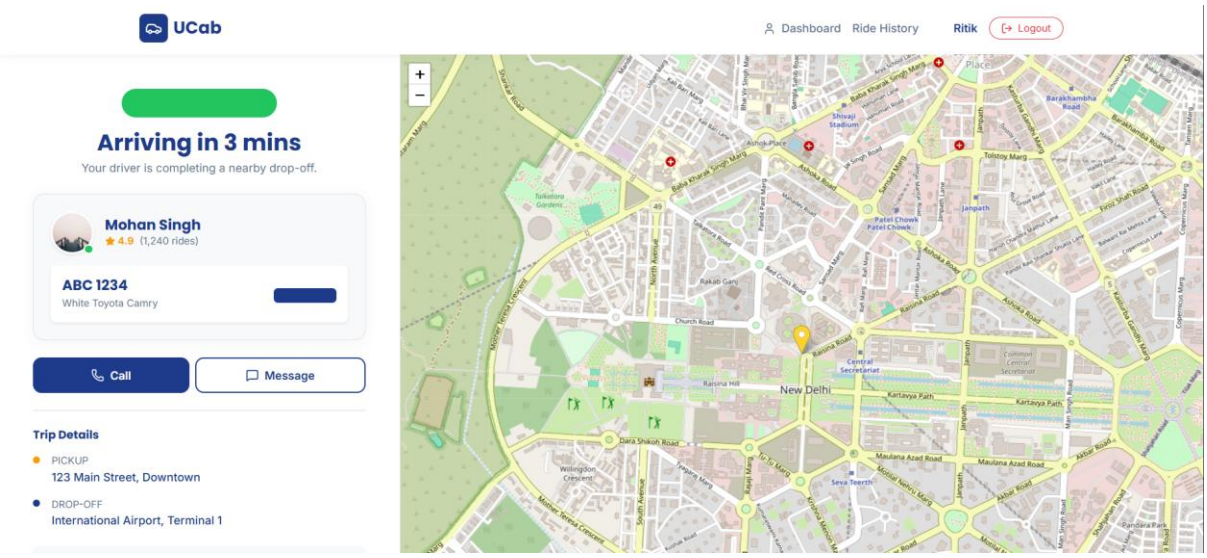
[: Ride Booking & Cab Selection Page]



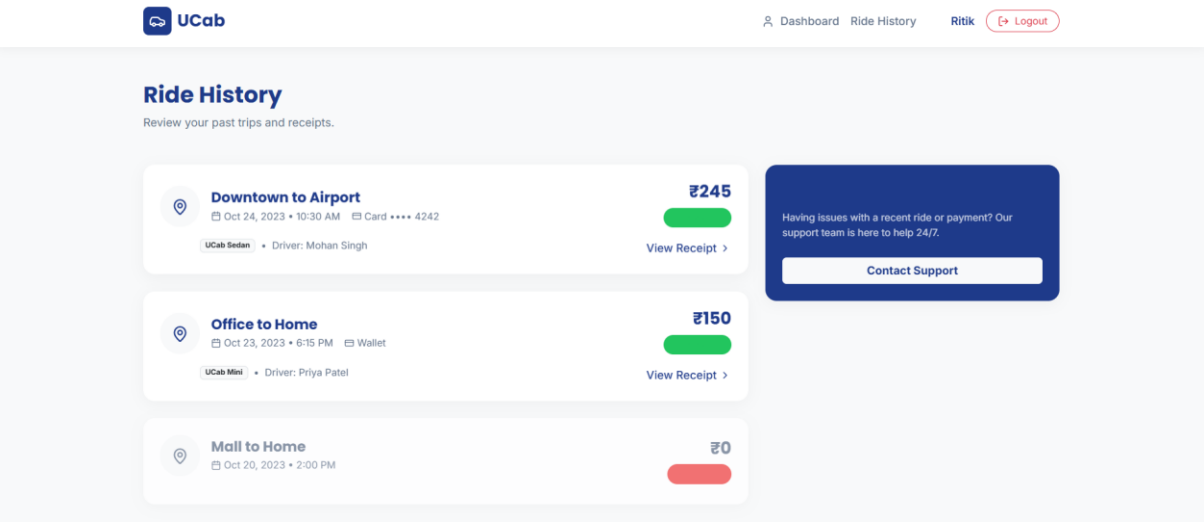
[QR Code Payment Interface]



[Ride Tracking & Leaflet Map Interface]



[User Dashboard / Ride History]



8. ADVANTAGES & DISADVANTAGES

Advantages:

- **Cost-Effective: Uses Leaflet.js instead of paid Google Maps APIs.**
- **Zero Middleman Fees: Direct P2P UPI payments via QR code eliminate payment gateway commissions.**
- **Real-Time Ready: Socket.io infrastructure provides a solid foundation for live tracking.**

- **Highly Scalable: Stateless JWT authentication allows the backend to handle multiple concurrent users easily.**

Disadvantages:

- **Manual Payment Verification: The static QR code requires the user to manually click "Payment Completed" as there is no webhook to verify the transaction automatically.**
- **Simulated Matching: The current MVP simulates the driver matching process rather than using actual geospatial radius queries.**

9. CONCLUSION

The UCab project successfully demonstrates a functional, scalable, and cost-effective prototype for a modern ride-sharing platform. By leveraging the MERN stack, Socket.io, and open-source tools like Leaflet.js, the project proves that high-quality, real-time applications can be built without relying heavily on expensive proprietary APIs. It successfully meets all its defined functional requirements for the MVP phase.

10. FUTURE SCOPE

- **Actual Payment Gateway Integration: Integrating Razorpay or Stripe with webhooks for automated payment verification.**
- **Geospatial Driver Matching: Implementing MongoDB 2dsphere indexes to match riders with the exact nearest driver based on live GPS coordinates.**
- **Push Notifications: Adding Firebase Cloud Messaging (FCM) to notify users when the app is running in the background.**
- **Google Maps API: Upgrading from Leaflet.js to Google Maps for highly accurate ETA and traffic-based routing.**

11. APPENDIX

- **Source Code: https://drive.google.com/drive/folders/1zVU2FHjvORf6ZkbA8dN4SeGuyfOeHS-H?usp=sharing**
- **Video Demo : https://drive.google.com/file/d/1N41Uc-9EW7nuZxOrb2SJmDyZj_Dz9Brx/view?usp=sharing**
- **GitHub Link: https://github.com/Ritikyadav2004/Ucab-Project**